



# **Universidad Nacional Autónoma de México**

**Matemáticas Aplicadas y Computación**

**Temas Selectos de Computacion**

**Practica 2**

**Profesor**

Eduardo Eloy Loza Pacheco

**Alumno**

Hernández Espinoza Gabriel Emanuel

# ÍNDICE

|                              |          |
|------------------------------|----------|
| <b>Objetivo</b>              | <b>2</b> |
| <b>Materiales y Métodos</b>  | <b>2</b> |
| <b>Resumen</b>               | <b>2</b> |
| <b>Antecedentes Teóricos</b> | <b>3</b> |
| <b>Desarrollo</b>            | <b>4</b> |
| <b>Conclusiones</b>          | <b>6</b> |
| <b>Referencias</b>           | <b>7</b> |

# Objetivo

Desarrollar una aplicación sencilla en iOS mediante Xcode que permita mostrar imágenes de forma secuencial a través de un botón 'Siguiente' y revelar el nombre asociado a cada imagen mediante un botón 'Mostrar'.

## Materiales y Métodos

Para la realización de esta práctica se utilizó una computadora Mac con el sistema operativo macOS y el entorno de desarrollo integrado Xcode. Se creó un proyecto tipo 'App' para iPhone, en el cual se emplearon los componentes de interfaz gráfica UIImageView, UILabel y UIButton. El código fue implementado en Swift, donde se utilizó un arreglo de nombres de imágenes para simular una galería. Se realizaron conexiones entre la interfaz gráfica y el código fuente mediante @IBOutlet y @IBAction, garantizando así la interacción entre el usuario y la aplicación.

## Resumen

En esta práctica se desarrolló una aplicación básica para iOS utilizando el entorno Xcode con el objetivo de comprender el manejo de interfaces gráficas y la interacción con el usuario. La aplicación consiste en un visor de imágenes donde, al presionar un botón denominado 'Siguiente', se despliega de manera secuencial una nueva imagen en pantalla. Adicionalmente, mediante un botón 'Mostrar', se revela el nombre de la imagen en un componente UILabel. El funcionamiento de la aplicación se basa en un arreglo que contiene los identificadores de las imágenes cargadas dentro de los recursos del proyecto, así como un índice que controla la posición actual.

Durante el desarrollo, se implementaron conceptos fundamentales de la programación en Swift como el manejo de arreglos, condicionales y métodos para la actualización de la interfaz. Asimismo, se practicó el uso de las conexiones entre Interface Builder y el código mediante las etiquetas @IBOutlet y @IBAction, indispensables para vincular la lógica de programación con los elementos gráficos. El resultado fue una aplicación funcional que refuerza los principios básicos del desarrollo de aplicaciones móviles y sienta las bases para la construcción de proyectos más avanzados en el ecosistema iOS.

# Antecedentes Teóricos

El desarrollo de aplicaciones móviles interactivas implica no solo la construcción de interfaces gráficas, sino también la integración de lógica que permita la manipulación dinámica de los datos y su representación visual. En el caso de iOS, el entorno de desarrollo Xcode junto con el lenguaje Swift ofrece al programador un marco de trabajo que favorece la implementación de estructuras de datos y mecanismos de control directamente asociados con la interfaz del usuario [1].

Un concepto central en este tipo de aplicaciones es el uso de arreglos (arrays), los cuales permiten almacenar colecciones de elementos de manera ordenada y acceder a ellos mediante índices. En Swift, los arreglos constituyen una de las estructuras más utilizadas debido a su eficiencia y facilidad de manipulación. En la práctica, los arreglos posibilitan gestionar conjuntos de imágenes, textos u otros recursos que se despliegan en la aplicación, siendo un elemento fundamental en el visor de imágenes desarrollado [2].

La navegación secuencial entre elementos se apoya en la lógica de control de flujo, particularmente en condicionales y ciclos. Estas estructuras permiten definir comportamientos dinámicos, como reiniciar el índice al alcanzar el final del arreglo, garantizando un ciclo continuo en la aplicación. El uso de estas herramientas no solo facilita el diseño de aplicaciones más intuitivas, sino que también refuerza el aprendizaje de principios básicos de programación.

Otro aspecto relevante es la vinculación entre la lógica y la interfaz gráfica mediante las anotaciones `@IBOutlet` y `@IBAction`. Estas conexiones constituyen la base del patrón Modelo-Vista-Controlador (MVC), donde la vista corresponde a los elementos gráficos (`UILabel`, `UIButton`, `UIImageView`), el controlador administra las interacciones y el modelo define los datos que se manipulan. Esta separación de responsabilidades mejora la organización del código y favorece la escalabilidad de la aplicación [3].

En conjunto, los antecedentes mencionados permiten comprender cómo estructuras de datos, control de flujo y principios de diseño de software se integran en la construcción de interfaces interactivas para dispositivos iOS.

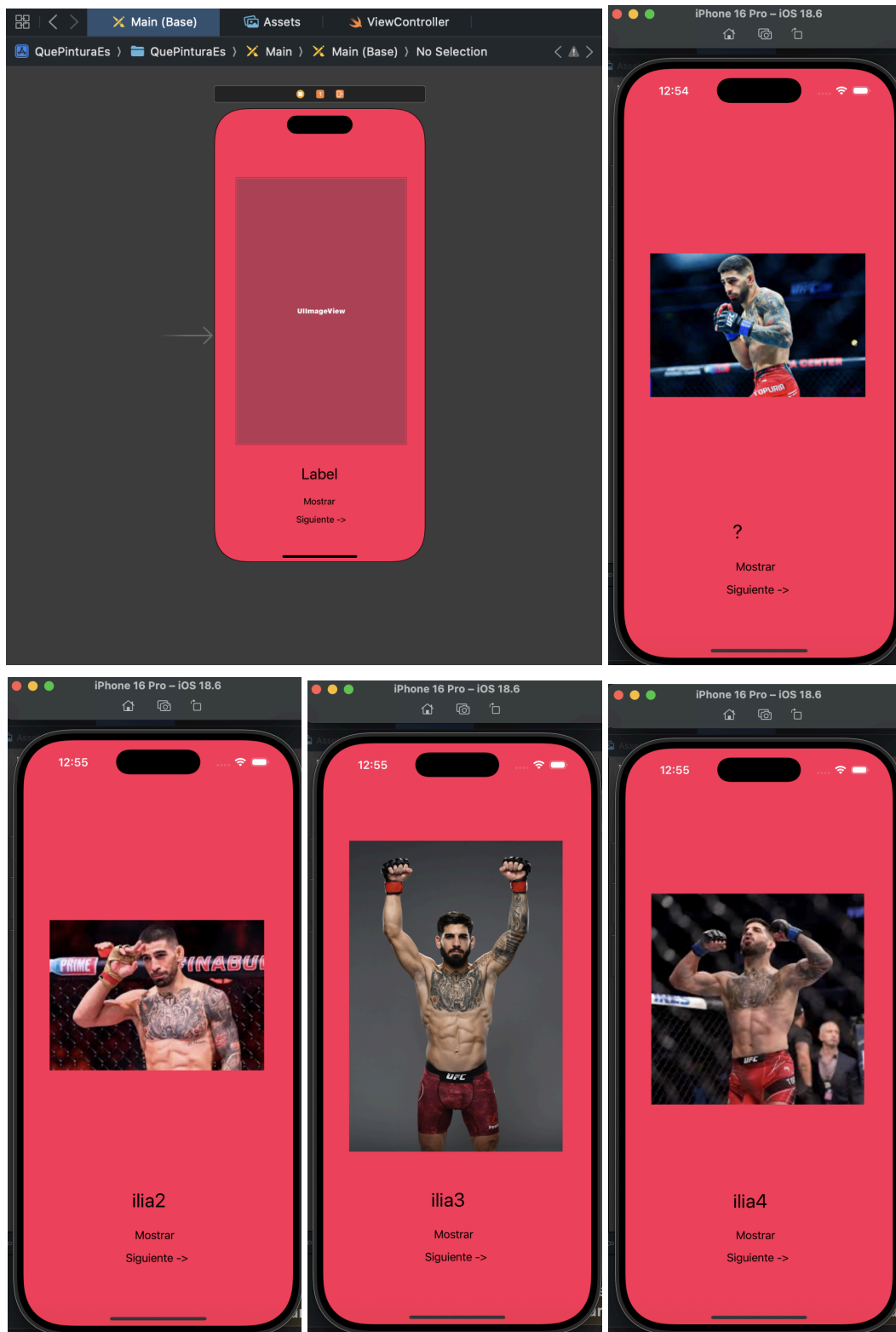
# Desarrollo

La práctica comenzó con la creación de un proyecto en Xcode, seleccionando la plantilla 'App' para dispositivos iPhone. Dentro del storyboard principal, se añadieron los elementos UILabel, UIImageView y dos botones. El primero de los botones fue configurado con la etiqueta 'Siguiente ->' y el segundo con la etiqueta 'Mostrar'. Estos elementos fueron distribuidos en la interfaz de manera que la imagen ocupara la parte central de la pantalla, mientras que la etiqueta y los botones se colocaron en la parte inferior para una interacción intuitiva.

El código en Swift incluyó un arreglo denominado `listaDeElementos` que contiene los nombres de las imágenes cargadas en los recursos del proyecto. Además, se declaró una variable `elementosIndice` que funciona como puntero a la posición actual dentro del arreglo. En la función `viewDidLoad()` se inicializó la primera imagen mediante la invocación del método `actualizarElemento()`.

La lógica del botón 'Siguiente' fue implementada en el método `@IBAction siguiente()`. Este incrementa el índice actual y, en caso de superar el número de elementos, reinicia el conteo a cero, garantizando un ciclo continuo en la galería de imágenes. Por su parte, el botón 'Mostrar' fue configurado para revelar el nombre del elemento actual en el UILabel, lo cual se implementó dentro del método `mostrar()`.

El método `actualizarElemento()` se encargó de asignar a la vista UIImageView la imagen correspondiente al índice actual y restablecer la etiqueta con un signo de interrogación, simulando el efecto de ocultar el nombre hasta que el usuario decida presionar 'Mostrar'. Esta dinámica permitió integrar funcionalidad interactiva básica, consolidando el entendimiento de la comunicación entre componentes visuales y lógica de programación.



Finalmente, la aplicación fue ejecutada en el simulador de iPhone, confirmando que tanto la rotación de imágenes como la visualización de los nombres funcionaban de manera correcta. Con ello se alcanzó el objetivo de la práctica.

# Conclusiones

La práctica permitió afianzar conocimientos sobre la creación de interfaces gráficas en Xcode y su integración con el lenguaje Swift. A través de la implementación de un visor de imágenes interactivo, se logró comprender el uso de arreglos, ciclos y condicionales para el manejo de datos, así como la importancia de las conexiones entre Interface Builder y el código mediante @IBOutlet y @IBAction.

El resultado fue una aplicación sencilla pero funcional, que evidencia el potencial de Xcode para el diseño rápido de prototipos interactivos. Además, el ejercicio sirvió para reforzar las bases del patrón MVC y para reflexionar sobre la importancia de la usabilidad en el diseño de interfaces. Estos aprendizajes resultan esenciales para abordar proyectos de mayor complejidad en el ámbito del desarrollo móvil.

Un aspecto relevante de la práctica fue la necesidad de garantizar que la aplicación funcionará de manera cíclica al recorrer los elementos del arreglo, lo cual permitió comprender de forma más profunda la lógica de control en Swift. De igual manera, la vinculación entre el código y la interfaz evidenció la importancia de mantener una buena organización y claridad en la estructura del programa.

Esta experiencia constituyó un paso importante en el dominio de las herramientas fundamentales para la programación en iOS. La práctica no solo proporcionó bases técnicas, sino también un enfoque metodológico para enfrentar futuros retos de desarrollo móvil, consolidando una base sólida para proyectos académicos y profesionales.

# REFERENCIAS

[1] Apple, Xcode Overview. Apple Developer, 2023. Disponible en:  
<https://developer.apple.com/xcode/>

[2] Apple, Swift Language Guide. Apple Developer Documentation, 2023. Disponible en:  
<https://docs.swift.org/swift-book/LanguageGuide/CollectionTypes.html>

[3] Apple, Human Interface Guidelines. Apple Developer, 2022. Disponible en:  
<https://developer.apple.com/design/human-interface-guidelines/>